

Capítulo 5

Implementación

5.1. Software utilizado

Para el desarrollo del proyecto, se han empleado diversas tecnologías de software que desempeñan roles fundamentales en el proceso de creación y ejecución. A continuación, se describen detalladamente cada una de estas tecnologías.

- Git/Github
- MySQL
- Postman
- Visual Studio Code
- Docker

5.1.1. Github

Git es un sistema de control de versiones distribuido que fue diseñado por Linus Torvalds con un enfoque en la eficiencia, confiabilidad y compatibilidad en el mantenimiento de versiones de aplicaciones con un gran número de archivos de código fuente. Su propósito principal es llevar un registro preciso de los cambios realizados en los archivos y coordinar el trabajo entre múltiples colaboradores que contribuyen a un repositorio compartido de código. Algunas de las características clave de Git incluyen la capacidad de trabajar de manera descentralizada, la gestión eficiente de ramas (branching) y fusiones (merges), así como la capacidad de revertir cambios o revisar el historial de versiones de manera fácil.

GitHub, por otro lado, es una plataforma de desarrollo colaborativo basada en la nube que utiliza el sistema de control de versiones Git. Permite alojar proyectos de software de manera centralizada y proporciona herramientas adicionales para la colaboración efectiva.

Los proyectos alojados en GitHub pueden ser públicos o privados, lo que significa que el código fuente puede ser accesible para todos o restringido a un grupo específico de colaboradores autorizados. Además de ser un repositorio para el código fuente, GitHub ofrece funciones como seguimiento de problemas (issue tracking), solicitudes de extracción (pull requests), wikis y otras herramientas que facilitan la gestión y colaboración en proyectos de desarrollo de software.

5.1.2. MySQL

MySQL es un sistema de gestión de bases de datos relacional de código abierto. Se utilizó MySQL para gestionar la base de datos implementada para el almacenamiento de los datos de entrada que se ingresan desde la interfaz, así como los resultados obtenidos luego de ejecutar la simulación.

5.1.3. Postman

Postman es una herramienta que ayuda a diseñar, desarrollar y probar plataformas de API (Application Programming Interfaces). Esta herramienta sirvió para probar conexiones al backend y comprobar su correcta funcionalidad sin la necesidad de tener construida una interfaz gráfica. Se realizaron peticiones con los métodos GET para la consulta de datos, PUT para la actualización de datos y POST para el ingreso de nuevos datos.

5.1.4. Visual Studio Code

Visual Studio Code es un editor de texto gratuito optimizado para el desarrollo y debugging de aplicaciones web modernas. Se utilizó para el desarrollo del código tanto para el frontend como para el backend, además de presentar una integración con MySQL y Git, generando un ambiente de desarrollo completo y estable. La integración con las tecnologías mencionadas ayudó a que el desarrollo de la aplicación se lograra de manera ágil.

5.1.5. Docker

Docker es una tecnología de código abierto que automatiza el despliegue de aplicaciones dentro de contenedores de software, lo que agrega una capa adicional de abstracción y automatización de virtualización de aplicaciones en múltiples sistemas operativos. Además, utiliza características de aislamiento de recursos del kernel Linux, lo que permite que estos contenedores sean independientes y se puedan ejecutar dentro de una sola instancia de Linux, evitando la sobrecarga de iniciar y mantener máquinas virtuales. La ventaja de esto

es que posee bastante flexibilidad, ya que se pueden crear, implementar, copiar y mover de un entorno a otro.

5.2. Hardware utilizado

Para el desarrollo de este proyecto, se empleó un equipo personal con las siguientes especificaciones de componentes:

Característica	Especificación
Procesador	Intel(R) Core(TM) i7-7700HQ CPU @ 2.80GHz
RAM	16,0 GB
Disco	1 TB
Tarjeta de Video	Nvidia GeForce GTX 1070

Tabla 5.1: Hardware Utilizado - Desarrollo.

Para la etapa de implementación del proyecto, este será implementado en un equipo que se encontrará en la Escuela de Informática de la Universidad de Valparaíso. El equipo contiene las siguientes características:

Característica	Especificación
Procesador	Intel(R) Core(TM) i9-12900KF @ 5.20 GHz
RAM	32.00 GB @ 3200 MHz
Disco HDD	8 TB
Disco SSD	1 TB
Tarjeta de Video	Nvidia RTX 3090 24 GB

Tabla 5.2: Hardware Utilizado - Implementación.

5.3. Lenguajes de programación

Dado que se está continuando con un proyecto ya desarrollado, se seguirá trabajando con los mismos lenguajes, y se explicará el por qué se decidió usarlos desde un principio. Comenzando por el lenguaje Python, que fue utilizado para desarrollar el backend. Este se tomó como base dado que inicialmente el código de propagación de Redman A. fue escrito en ese lenguaje. Luego se utilizó JavaScript, específicamente una biblioteca de JavaScript llamada ReactJS, que se utilizó para el desarrollo del frontend. A continuación, se describen resumidamente cada uno de estos lenguajes utilizados.

5.3.1. Python

Python es un lenguaje de programación interpretado, multiparadigma, que soporta parcialmente la orientación a objetos, programación imperativa y funcional. Como se mencionó anteriormente, se utilizó Python para el desarrollo del backend del proyecto debido a que el código de propagación de Reidmen A. inicialmente fue escrito en este lenguaje. Al desarrollar utilizando el mismo lenguaje, facilitó el llamado desde el backend al código de simulación, además de facilitar la conexión con la base de datos utilizando librerías y modificando en pequeña escala el código de propagación para poder realizar una correcta conexión.

5.3.2. JavaScript

JavaScript comúnmente abreviado como JS, es un lenguaje de programación interpretado. Se define como un lenguaje orientado a objetos, basado en prototipos, imperativo, débilmente tipado y dinámico. Se utiliza principalmente del lado del cliente, generalmente ejecutado por un navegador web, permitiendo mejoras en la interfaz de usuario y páginas web dinámicas. También se puede ejecutar JavaScript del lado del servidor, Server-side JavaScript o SSJS. Para el desarrollo del frontend del proyecto, se optó por utilizar una biblioteca de código abierto de JavaScript llamada React [52], también conocida como React.js o ReactJS. Esta fue diseñada con el objetivo de facilitar el desarrollo de interfaces de usuario de aplicaciones en una sola página, intentando ayudar a los desarrolladores a construir aplicaciones que utilizan datos que cambian todo el tiempo. Su objetivo principal es ser sencillo, declarativo y fácil de combinar.

5.4. Estrategia de implementación

El patrón de diseño se mantendrá en el que fue definido anteriormente Mediator, el cual juega un papel crucial en la reducción de dependencias entre objetos al eliminar conexiones innecesarias o inapropiadas. En el contexto de este sistema, el patrón Mediator opera como una especie de "torre de control" frente a los componentes, especialmente en las vistas del frontend desarrolladas con React.

En el frontend, al trabajar con React, se busca modularizar los componentes para mejorar la mantenibilidad y flexibilidad del código. El patrón Mediator permite gestionar las interacciones entre estos componentes, evitando que tengan dependencias directas entre sí. Cada componente se comunica a través del mediador, lo que favorece la cohesión y la independencia entre ellos. Esto es particularmente útil al considerar la naturaleza modular y dinámica de las interfaces de usuario en React.

Además, el patrón Mediator se extiende a la integración del código de propagación de Reidmen A. En este caso, cada componente o módulo de simulación puede funcionar de

manera independiente, dependiendo solo de los parámetros proporcionados durante la ejecución de la simulación. La reducción de dependencias innecesarias mejora la flexibilidad y la facilidad de mantenimiento de todo el sistema.

En el ámbito de la integración con la base de datos, el patrón Mediator desempeña un papel esencial al dirigir la comunicación entre la plataforma web (que incluye el código de simulación) y la base de datos. El mediador actúa como un intermediario eficiente, asegurando una comunicación ordenada y desacoplada entre estos dos componentes críticos del sistema.

En resumen, la implementación del patrón Mediator en este proyecto contribuye significativamente a la gestión de dependencias, promoviendo la modularidad y la independencia entre los componentes, tanto en el frontend con React como en la integración de la simulación y la base de datos.

Para este proyecto, se plantean dos iteraciones importantes. La primera iteración tiene como objetivo analizar el estado actual del proyecto, corregir errores existentes y verificar los resultados de la simulación. Durante esta fase inicial, se busca una comprensión exhaustiva de la situación actual del proyecto, abordando posibles problemas y asegurando que los resultados de la simulación sean precisos y confiables.

Para la segunda iteración, el objetivo es implementar las nuevas funcionalidades requeridas, incluyendo los requisitos funcionales mencionados en la sección 3.5.1. Durante esta etapa, se centrará en la expansión y mejora del sistema, incorporando las características y capacidades específicas identificadas como necesarias según los requisitos establecidos. Se seguirá un enfoque iterativo para garantizar la calidad y la coherencia en la implementación de estas nuevas funcionalidades.

Ambas iteraciones están diseñadas para avanzar de manera ordenada y controlada, abordando primero los problemas existentes antes de introducir nuevas funcionalidades. Esto ayuda a minimizar riesgos y garantizar un desarrollo y una implementación más efectivos y eficientes en el proyecto.

Capítulo 6

Pruebas

Como se ha mencionado en la Sección 5.4, para esta primera iteración nos centramos en verificar el estado actual del proyecto y corregir los errores encontrados. Con este propósito, se llevaron a cabo pruebas unitarias para cada una de las entidades que consisten en el código de propagación de Reidmen, el backend y el frontend.

6.1. Pruebas unitarias

6.1.1. Pruebas Unitarias backend

Para llevar a cabo las pruebas unitarias en la sección del backend, se empleó Postman. Durante este proceso de pruebas, se utilizaron solicitudes HTTP para interactuar con los puntos finales del backend, validando la correcta implementación de las funciones y la manejo adecuado de los datos.

Prueba	Salida Esperada	Estado
Request de simulaciones	Recibir todas las simulaciones que se encuentran en el sistema.	Ok
Request de detalle de una simulación	Recibir todos los datos del detalle de la simulación correspondiente.	Ok
Validaciones ingreso de parámetros	Desplegar mensajes de error desde el backend.	Ok
Prueba de guardado en BD - Parámetros	Guardar de manera correcta en la base de datos.	Ok
Prueba de guardado en BD - Resultados	Guardar resultados correctamente en la base de datos.	Ok
Generar petición descarga de archivo	Generar una respuesta para descargar el archivo correspondiente.	Ok

Tabla 6.1: Tabla de Pruebas Backend

6.1.2. Pruebas Unitarias frontend

Para esta sección, se realizaron pruebas de funcionalidad para cada una de las vistas, evaluando cada uno de los componentes que conforman una vista:

Prueba	Salida Esperada	Estado
Navegación formulario	Debe cargar la pantalla de ingreso de parámetros.	Ok
Navegación listado de simulaciones	Debe cargar la vista del listado de simulaciones.	Ok

Tabla 6.2: Tabla de Pruebas de Pantalla Principal

Prueba	Salida Esperada	Estado
Navegación Pantalla Principal	Debe volver a la página principal.	Ok
Enviar datos desde el formulario	Debe enviar los mismos valores ingresados para cada parámetro correspondiente.	Ok
Validaciones de error	Se debe mostrar un mensaje del campo ingresado erróneamente.	Ok.
Ventana de Información	Debe abrir correctamente la ventana de información de los parámetros.	Ok

Tabla 6.3: Pruebas Unitarias frontend - Pantalla Ingreso de Parámetros

Prueba	Salida Esperada	Estado
Navegación Ingreso de Parámetros	Debe navegar a la pantalla de ingreso de parámetros.	Ok
Ver Detalles de una simulación terminada	Mostrar correctamente la ventana para una simulación terminada con los detalles de la simulación correspondiente.	Ok
Ver Detalles de una simulación en ejecución	Mostrar correctamente la ventana para una simulación en ejecución con los detalles de la simulación correspondiente.	Ok
Descargar Archivo	Mostrar mensaje de descarga del archivo.	Ok

Tabla 6.4: Pruebas Unitarias frontend - Pantalla Listado de Simulaciones

6.1.3. Pruebas Unitarias Código de Propagación de Reidmen A

Durante las pruebas realizadas en la sección del código de propagación, los resultados inicialmente arrojaban un valor de 0.0. Como respuesta a este inconveniente, se llevaron a cabo modificaciones en las definiciones de las variables con el objetivo de obtener resultados más cercanos a los valores reales. Estas adaptaciones fueron esenciales para mejorar la precisión y confiabilidad de los resultados generados por el código de propagación, asegurando así una mayor coherencia con la realidad.

Prueba	Salida Esperada	Estado
Ingreso de Parámetros	Debe ejecutarse la simulación sin problemas.	Ok
Obtener Resultados	Debe entregar las gráficas y exportar el archivo .mat.	Ok
Verificar Distancia	Debe calcular correctamente la distancia total de la placa en base a la separación y distancias que poseen los sensores.	Ok
Resultados esperados	Debe entregar resultados aproximados a una medicion real	Ok

Tabla 6.5: Pruebas Unitarias Código de Propagación de Reidmen Arostica Sin Frecuencia

Prueba	Salida Esperada	Estado
Ingreso de Parámetros	Debe ejecutarse la simulación sin problemas.	Ok
Obtener Resultados	Debe entregar las gráficas y exportar el archivo .mat.	Ok
Verificar Distancia	Debe calcular correctamente la distancia total de la placa en base a la separación y distancias que poseen los sensores.	Ok
Resultados esperados	Debe entregar resultados aproximados a una medicion real	Ok

Tabla 6.6: Pruebas Unitarias Código de Propagación de Reidmen Arostica Con Frecuencia

6.2. Pruebas de integración

La prueba de integración consiste en verificar el correcto funcionamiento del conjunto de entidades del sistema. Para llevar a cabo esta evaluación, para esto se realizo un recorrido que comenzó desde el ingreso de parámetros, pasando por el despliegue del listado de simulaciones, para luego iniciar la simulación y descargar el archivo .mat resultante.

Durante esta prueba, no se identificaron nuevos problemas que no hubieran sido detectados anteriormente en las pruebas unitarias. Por lo tanto, se puede concluir que, de manera local, el sistema presenta el funcionamiento esperado.

6.3. Pruebas de Sistema

Para la prueba de sistema en este caso, se siguió el mismo flujo utilizado en la prueba de integración. Sin embargo, al ejecutar el proyecto desde un contenedor Docker, se enfrentaron varios desafíos. Inicialmente, el proyecto no se ejecutó correctamente debido a un error al intentar descargar las dependencias. Este problema se debió a que la última versión de Flask requería de otra dependencia llamada Werkzeug, la cual no estaba especificada en el archivo de dependencias para su descarga.

Una vez resuelto este problema, surgió otro inconveniente relacionado con la creación de la base de datos necesaria para el sistema. Aunque el script intentaba crear la base de datos, no generaba la tabla necesaria para almacenar las simulaciones ingresadas. Como resultado, no era posible agregar simulaciones al sistema. Para solventar esto, se debió crear manualmente la tabla, garantizando así el correcto funcionamiento del sistema.

Posteriormente, al ejecutar una simulación, se observó que el valor de los sensores detectado por la simulación era siempre igual a 0.0, tanto en simulaciones con frecuencia como sin ella. Este problema llevó a la identificación de variables mal definidas en

el código, las cuales se corrigieron para obtener resultados más precisos y cercanos a las mediciones reales.

Con estos ajustes implementados, el sistema empezó a arrojar resultados más coherentes y aproximados a las expectativas, demostrando así un mejor funcionamiento después de abordar y resolver cada uno de los problemas identificados durante la prueba de sistema.